

Packer - Custom AMI / Machine Image Automation

Building Immutable, Golden Machine Images with HashiCorp Packer - Interview Preparation Guide

Covers: why teams bake custom machine images instead of configuring servers by hand, Packer's core concepts (templates, builders, provisioners, post-processors), the end-to-end image-build pipeline, a real HCL template for building a custom Ubuntu AMI on AWS, and a step-by-step rebuild of a Packer + GitHub Actions CI/CD pipeline that produces and verifies a custom AMI.

Contents

1. Why Packer? Immutable Infrastructure & Golden Images
2. Core Concepts: Templates, Builders, Provisioners, Post-Processors
3. How a Packer Build Actually Runs
4. Automating Image Builds Across Platforms (CI/CD)
5. Packer Configuration Files & a Real HCL Template
6. Packer CLI Commands
7. Step-by-Step Hands-On Rebuild
8. Interview Q&A Bank

1. Why Packer? Immutable Infrastructure & Golden Images

What Packer is: Packer is an open-source HashiCorp tool for creating machine images - AMIs, Docker images, VMware templates, Azure Managed Images, Google Compute images, and more - from a single, version-controlled configuration file. Instead of launching a server and configuring it by hand (or configuring it after every boot with a tool like Ansible), you bake the configuration *into the image itself* once, then launch identical instances from that image every time.

What Packer gives you:

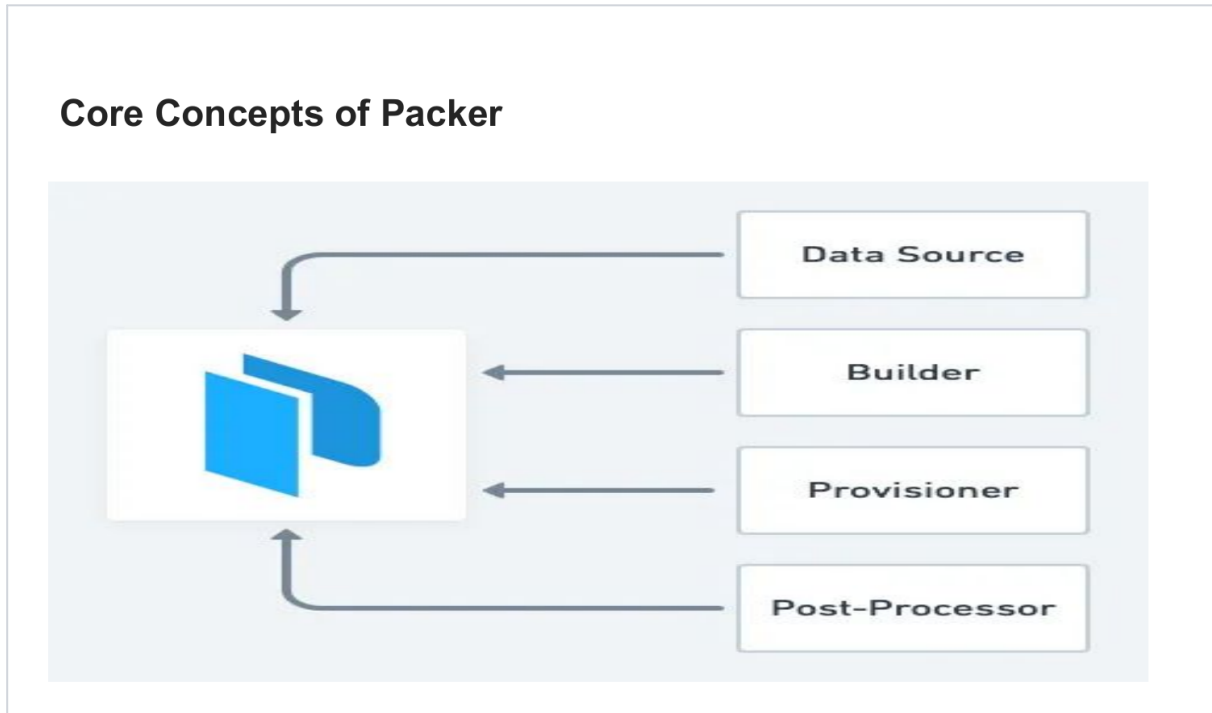
- **Consistent image creation** - the machine configuration is defined as code, so the same image is built the same way every single time (no configuration drift between servers).
- **Automated builds** - Packer can build images for multiple platforms (AWS, Azure, GCP, VMware) from a single configuration file in one run.
- **Reproducible environments** - every environment provisioned from the image is identical, which makes deployments predictable and rollbacks trivial (just relaunch the previous image version).

Why this matters for real projects:

- Packer is a genuinely differentiating skill - it is easy to get started with, and custom base images built this way are used across the industry, in companies of every size.
- A common team split: engineers early in their career typically start with a **shell script** as the provisioner (simple, few dependencies); more experienced engineers move to an **Ansible-based** provisioner for larger, more structured configuration.
- The end goal is almost always a **Packer-based pipeline** in whichever CI/CD tool your team uses - taking a cloud-provided base image (e.g. Ubuntu on AWS/Azure) and installing your required software on top of it via a shell script or Ansible provisioner, producing a golden image that later gets referenced from Terraform or a launch template.
- This directly extends the multi-account AWS foundation from the Landing Zone module: a custom AMI built here is exactly the kind of artifact a dedicated tooling/shared-services account would publish for the workload accounts to consume.

2. Core Concepts: Templates, Builders, Provisioners, Post-Processors

Every Packer build is driven by a **template** (an HCL or JSON configuration file) that wires together four kinds of building blocks.



The four inputs that feed a Packer build: a Data Source (e.g. lookup the latest base AMI), a Builder (which platform to build for), a Provisioner (how to configure the machine), and a Post-Processor (what to do with the finished image).

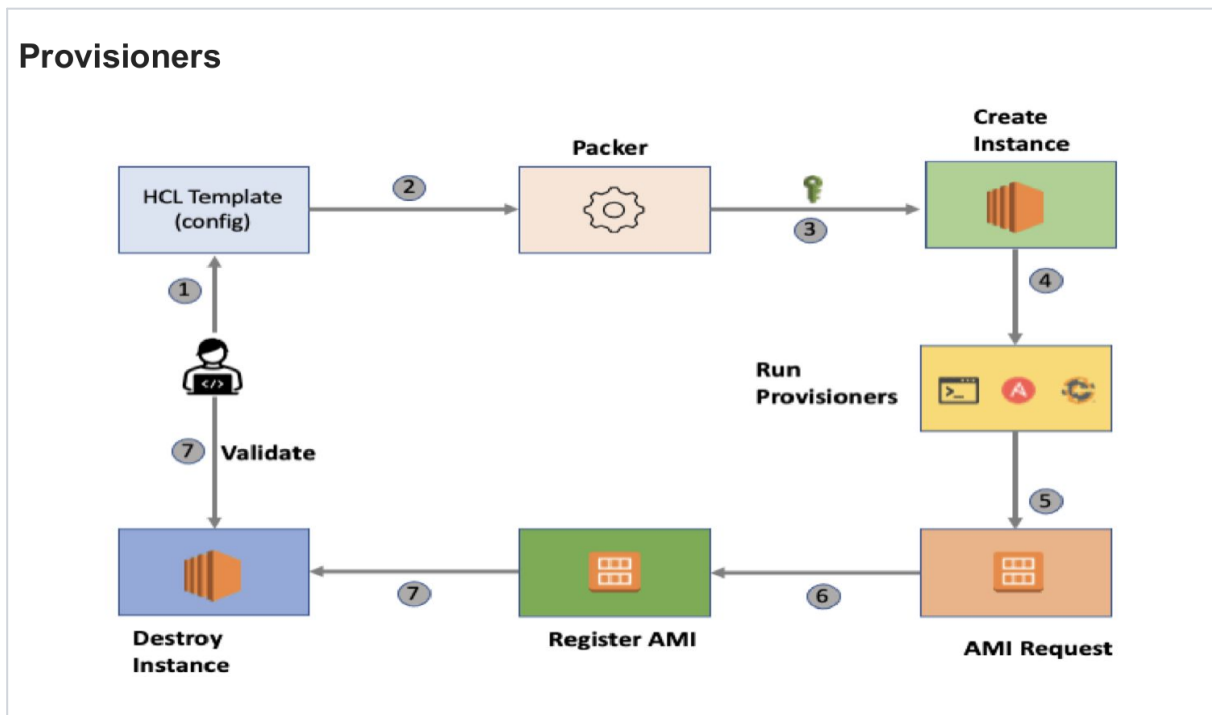
- **Data source** - reads information at build time (for example, looking up the latest base AMI ID matching a filter) so the template does not need to hardcode values that change over time.
- **Builder** - defines *where* and *how* the image gets built: which platform (amazon-ebs, azure-arm, googlecompute, ...), which base/source image, instance type, and region. Packer launches a temporary instance from the builder's settings.
- **Provisioner** - defines *what gets installed or configured* on that temporary instance before it is captured as an image - a shell script, an Ansible playbook, or similar.
- **Post-processor** - runs after the image is built, e.g. compressing the artifact, uploading it to an artifact repository, or tagging/copying it to additional regions or accounts.

Summary - how source and build fit together:

- The source block specifies the builder type (such as amazon-ebs or googlecompute) and a unique name for that source. It holds all the settings needed to interact with that platform - region, instance type, base image, credentials, and more.
- The build block tells Packer what to *do* with that source, including which provisioner scripts to run and any post-processing actions.

3. How a Packer Build Actually Runs

On AWS specifically, a Packer build follows a strict launch -> configure -> capture -> destroy sequence - Packer never leaves the temporary build instance running.

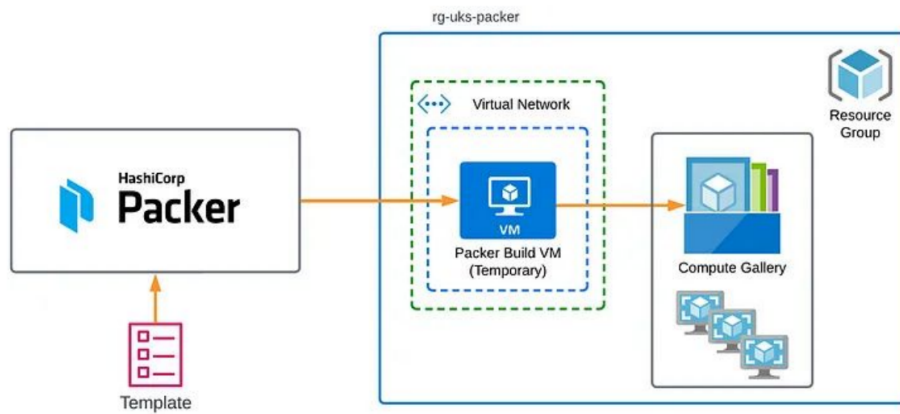


The full build/provision/validate pipeline: you write the HCL template, Packer creates a temporary EC2 instance, runs the configured provisioners against it, requests and registers an AMI from it, then destroys the instance and the AMI is validated back against the template.

- 1 You author the HCL template (the config) - this is the only artifact you keep in source control.
- 2 Packer reads the template and launches a temporary instance from the configured builder, using a generated temporary key pair.
- 3 Packer connects to the instance and runs every configured **provisioner** in order (shell script, Ansible, etc.) - this is where your software actually gets installed.
- 4 Once provisioning finishes, Packer requests an AMI from the now-configured instance.
- 5 AWS registers the new AMI (a point-in-time, bootable snapshot of the instance's root volume).
- 6 Packer terminates the temporary instance - it was only ever needed to build the image, not to run it.
- 7 The result is validated back against the template so you know the build succeeded and the final AMI ID is available to reference elsewhere (a launch template, an Auto Scaling group, Terraform, etc.).

On Azure, the same idea looks like this:

Builder

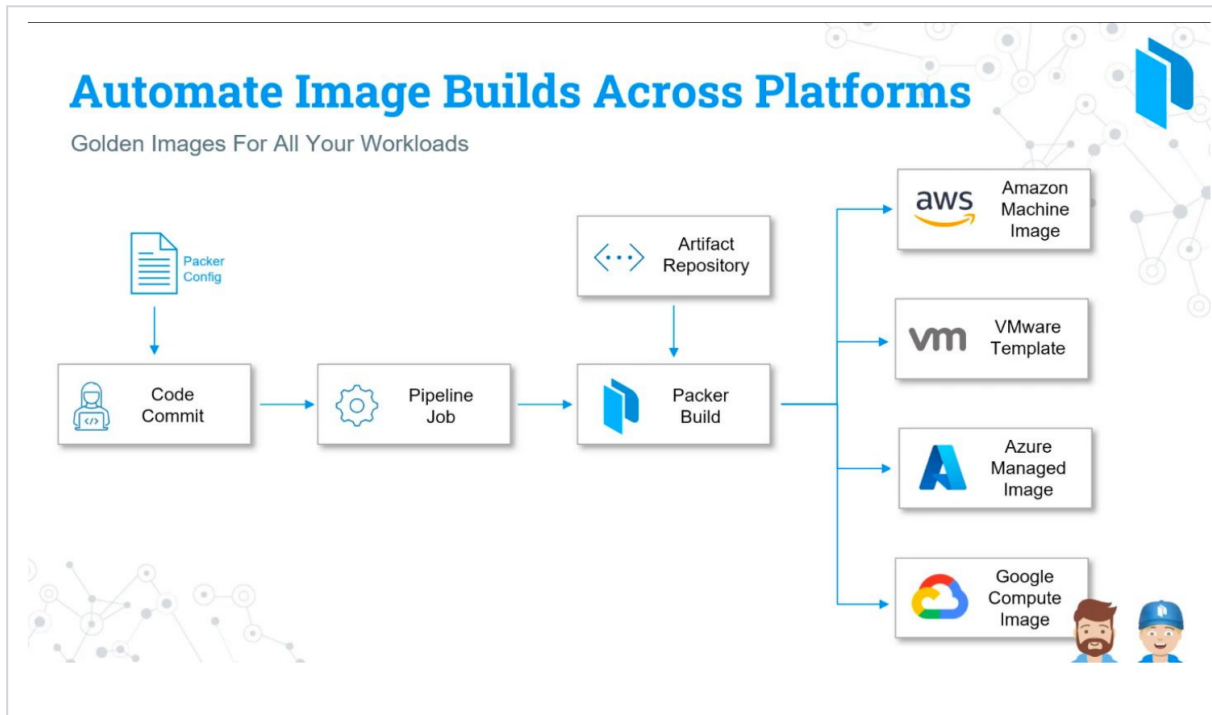


Azure equivalent: HashiCorp Packer reads the Template, spins up a temporary Packer Build VM inside a Virtual Network in a Resource Group, then publishes the finished, versioned image into a Compute Gallery for reuse.

The platform-specific names differ (Compute Gallery vs. AMI, Resource Group vs. an AWS account), but the underlying pattern is identical across every cloud Packer supports: build once inside an isolated, temporary compute resource, capture it as a reusable image, then tear the temporary resource down.

4. Automating Image Builds Across Platforms (CI/CD)

In practice, Packer builds are not run by hand - they are triggered from a CI/CD pipeline whenever the template or provisioning scripts change, so the golden image is always rebuilt from a known, auditable commit.



A code commit to the Packer config triggers a pipeline job; Packer Build reads any supporting scripts from an artifact repository and fans out to produce an Amazon Machine Image, a VMware Template, an Azure Managed Image, and a Google Compute Image from the same source configuration.

- A **code commit** to the Packer configuration (and any provisioning scripts) triggers a **pipeline job** in your CI tool of choice (GitHub Actions, Jenkins, GitLab CI, etc.).
- The pipeline job invokes **Packer Build**, which can pull supporting assets from an **artifact repository** as part of provisioning.
- A single Packer configuration can fan out to multiple target platforms in one build - producing an Amazon Machine Image, a VMware Template, an Azure Managed Image, and a Google Compute Image, all as "golden images" for their respective workloads.
- This is exactly the pattern demonstrated hands-on in Section 7: a GitHub Actions workflow that installs Packer, then runs `packer build` against a committed HCL template to produce a new AWS AMI on every push.

5. Packer Configuration Files & a Real HCL Template

- **JSON/HCL format** - Packer configuration files use JSON or HCL (HashiCorp Configuration Language) to define the image build process; HCL is the modern, recommended format.
- **Builder definition** - the source/builder section specifies the target platform (AWS, Azure, GCP, ...) and the necessary configuration options for that platform.
- **Provisioner definition** - the provisioner section defines the scripts or commands to be executed against the temporary instance during the image build process.

Real template - building a custom Ubuntu AMI on AWS

This is the actual working HCL template used in the demo, defining the required Amazon plugin and an amazon-ebs source that looks up the latest Ubuntu 22.04 (Jammy) image and builds a new, timestamped AMI from it:

```
packer {
  required_plugins {
    amazon = {
      source = "github.com/hashicorp/amazon"
      version = "~> 1"
    }
  }
}

source "amazon-ebs" "ubuntu" {
  ami_name      = "packer-ubuntu-aws-{{timestamp}}"
  instance_type = "t3.micro"
  region        = "us-west-2"
  source_ami_filter {
    filters = {
      name               = "ubuntu/images/*ubuntu-jammy-22.04-amd64-server-*"
      root-device-type   = "ebs"
      virtualization-type = "hvm"
    }
    most_recent = true
    owners      = ["099720109477"]
  }
  ssh_username = "ubuntu"
}
```

Reading this template for an interview:

- `required_plugins` pins the amazon builder plugin and its version constraint - Packer downloads/verifies this the first time you run `packer init`.
- `{{timestamp}}` is a Packer template function that expands to the current Unix timestamp, so every build produces a uniquely named AMI instead of colliding with the previous one.
- `source_ami_filter` with `owners = ["099720109477"]` pins the base image to Canonical's official Ubuntu AMI owner ID and always picks the `most_recent` match - this avoids hardcoding a specific AMI ID that will eventually be deprecated.

- A complete template also needs a `build` block referencing this source (`build { sources = ["source.amazon-ebs.ubuntu"] ... }`) with one or more `provisioner` blocks inside it - that is where the actual software installation (shell script or Ansible) is attached.

6. Packer CLI Commands

Command	What it does
<code>packer init</code>	Installs the plugins required by an HCL2 template (reads <code>required_plugins</code> and downloads them) so that the build can proceed
<code>packer validate</code>	Validates the syntax and configuration of a Packer template without building anything - the fast check
<code>packer build</code>	Builds one or more machine images from a Packer configuration file - this is the command that actually builds the images
<code>packer inspect</code>	Prints the components of a Packer configuration file (variables, builders, provisioners) without running them
<code>packer console</code>	Launches an interactive console for testing Packer expressions and configuration values before putting them into a template

Typical order in a pipeline: `packer init` -> `packer validate` -> `packer build`. Running `validate` as its own CI step before `build` catches template mistakes in seconds instead of after a temporary instance has already launched.

7. Step-by-Step Hands-On Rebuild

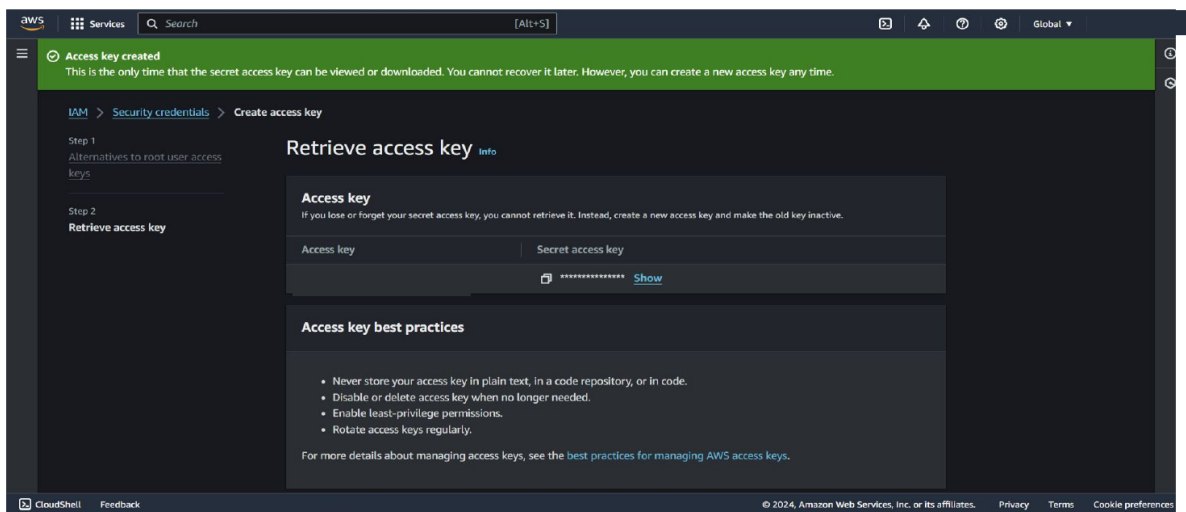
This rebuilds a small end-to-end pipeline: a GitHub repository holding a Packer HCL template, a GitHub Actions workflow that installs Packer and runs the build, and AWS credentials stored as repository secrets so the workflow can create a real AMI.

Step 1 - Set up the repository and template

- 1 Create (or fork) a repository containing an HCL Packer template such as the one in Section 5 (commonly named something like `aws-ubuntu.pkr.hcl`) plus a GitHub Actions workflow file under `.github/workflows/` that runs `packer init`, `packer validate`, and `packer build` against it.
- 2 Locally, install Packer and sanity-check the template before relying on CI: `packer init` . then `packer validate ..`

Step 2 - Create AWS access keys for the pipeline

Step2: Create aws access keys from aws console to create AMI



IAM -> Security credentials -> Create access key: the console shows the access key ID once, plus best practices (never commit it, disable/delete when unused, enable least privilege, rotate regularly).

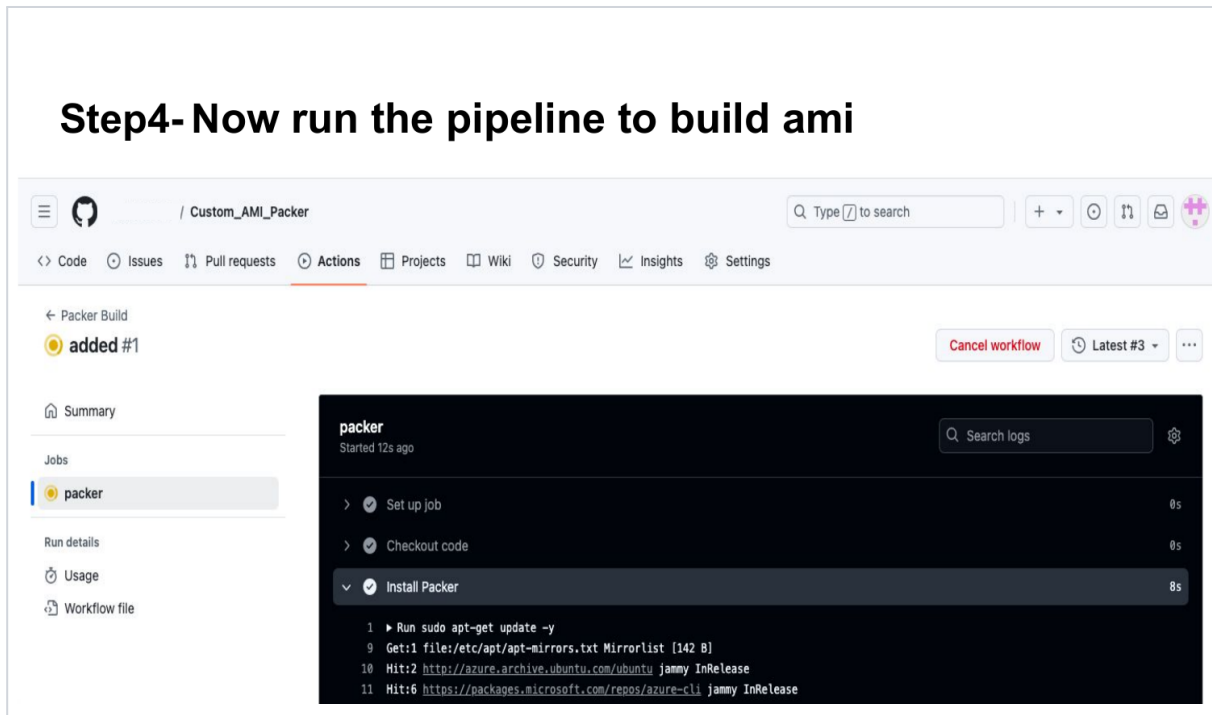
- 1 In the AWS Console, go to **IAM -> Security credentials -> Create access key** for a user (or role) that only has the permissions Packer needs to build and register an AMI - not a root/admin key.
- 2 Copy the access key ID and secret access key immediately - AWS only shows the secret once.
- 3 Follow the access-key best practices shown in the console: never store it in plain text or in a code repository, disable or delete it when it is no longer needed, keep it least-privilege, and rotate it regularly.

Step 3 - Store the credentials as GitHub repository secrets

- 1 In the repository, go to **Settings -> Secrets and variables -> Actions**.

- 2 Add two repository secrets, e.g. `AWS_ACCESS_KEY` and `AWS_SECRET_KEY`, using the values from Step 2.
- 3 The workflow file references these secrets as environment variables at build time so the raw keys never appear in the template or in the workflow logs.

Step 4 - Run the pipeline to build the AMI



The screenshot shows the GitHub Actions interface for a repository named 'Custom_AMI_Packer'. The main heading is 'Step4- Now run the pipeline to build ami'. The 'Actions' tab is selected, showing a workflow named 'Packer Build' with a status of 'added #1'. On the right, there are buttons for 'Cancel workflow' and 'Latest #3'. The left sidebar contains a 'Jobs' section with a 'packer' job selected. The main area displays the 'packer' job logs, which include the following steps and commands:

```
packer
Started 12s ago

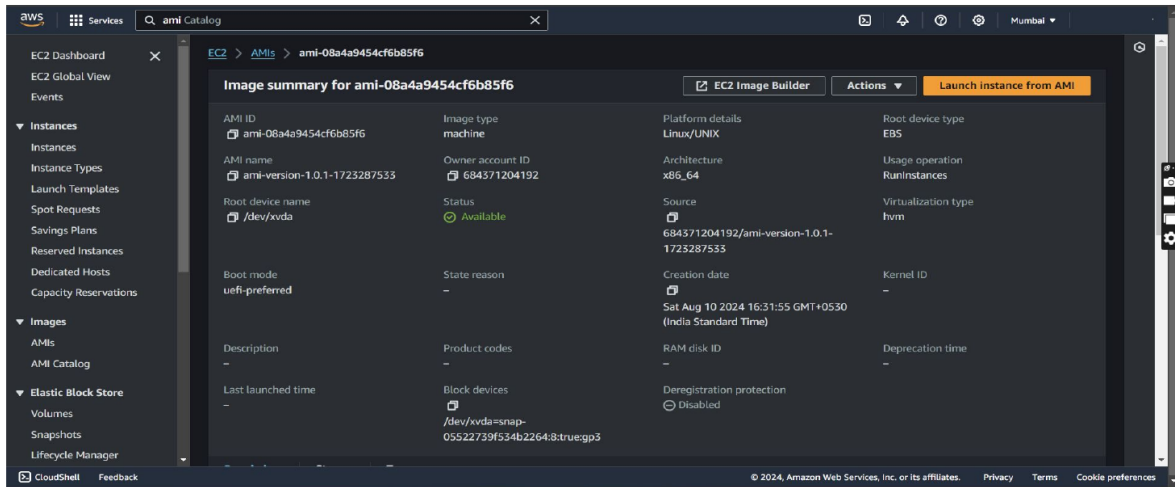
> Set up job 0s
> Checkout code 0s
> Install Packer 8s
  1 ▶ Run sudo apt-get update -y
  9 Get:1 file:/etc/apt/apt-mirrors.txt Mirrorlist [142 B]
 10 Hit:2 http://azure.archive.ubuntu.com/ubuntu jammy InRelease
 11 Hit:6 https://packages.microsoft.com/repos/azure-cli jammy InRelease
```

A GitHub Actions run of the Packer Build workflow: Set up job -> Checkout code -> Install Packer -> (then packer init / validate / build) - each step's logs are visible live in the Actions tab.

- 1 Push a commit (or manually trigger the workflow) so the GitHub Actions run starts.
- 2 Watch the job steps execute in order: setting up the job, checking out the code, installing Packer, then running `packer init`, `packer validate`, and finally `packer build`.
- 3 The build step launches a temporary EC2 instance, runs the configured provisioner, registers a new AMI, and terminates the instance - all inside the pipeline run, with no manual console work.

Step 5 - Verify the AMI in the AWS console

Step5: check the AMI from the aws console and launch an instance to check it correctly.



EC2 -> AMIs -> Image summary for the newly built AMI: status Available, architecture, root device, and creation date confirm the pipeline produced a valid, launchable image.

- 1 In the AWS console, go to **EC2 -> Images -> AMIs** and search for the AMI name your template generates (it will include the `{{timestamp}}` suffix).
- 2 Confirm the **Status** is **Available**, and check the architecture, root device type, and creation date match what you expect.
- 3 Click **Launch instance from AMI** to boot a real instance from it and confirm the software your provisioner installed is actually present - this is the real proof the build worked, not just that the pipeline turned green.

8. Interview Q&A Bank

Q: What problem does Packer solve, and why not just configure servers with a tool like Ansible at boot time?

A: Packer bakes configuration into the machine image itself instead of applying it every time a server boots. That gives you consistent, reproducible environments (every instance launched from the image is identical), much faster boot times in production (no lengthy configuration step at launch), and a versioned, auditable artifact (the AMI ID) you can roll forward or back. Ansible and Packer are complementary, not competing - Packer commonly calls Ansible as a provisioner during the image build itself.

Q: What are the core building blocks of a Packer template?

A: A data source (looks up values at build time, e.g. the latest base AMI), a builder/source (defines the target platform and how to launch the temporary build instance), a provisioner (defines what gets installed/configured on that instance - shell script or Ansible), and a post-processor (runs after the image is captured, e.g. uploading it to an artifact repository or copying it to other regions).

Q: Walk me through what actually happens when you run packer build.

A: Packer reads the template, launches a temporary instance from the configured source/builder using a generated key pair, connects to it and runs every configured provisioner in order, then requests and registers a new AMI from the now-configured instance. Once the AMI is registered, Packer terminates the temporary instance - it only ever existed to produce the image, never to serve traffic.

Q: What's the difference between the source block and the build block in an HCL Packer template?

A: The source block specifies the builder type (e.g. amazon-ebs) and a unique name for it, holding all the platform-specific settings - region, instance type, base image filter, credentials. The build block tells Packer what to actually do with that source: which provisioners to run against it and any post-processing steps, referencing the source by name.

Q: In the sample AWS template, why use a source_ami_filter with owners and most_recent instead of a hardcoded AMI ID?

A: Hardcoded AMI IDs are region-specific and eventually get deprecated by the base-image publisher. Filtering by name pattern, owner account ID (e.g. Canonical's official Ubuntu owner ID), and most_recent = true always resolves to the current, valid base image at build time, so the template keeps working without manual updates and without accidentally building from a stale or untrusted AMI.

Q: What does {{timestamp}} do in an ami_name, and why does it matter?

A: It's a Packer template function that expands to the current Unix timestamp at build time, so every build produces a uniquely named AMI. Without it, a second build with the same ami_name would fail or collide - unique, timestamped names are what let you keep a history of image versions and roll back to a previous one.

Q: What's the purpose of packer init, and when do you run it?

A: packer init reads the required_plugins block of an HCL2 template and downloads/installs the plugins it declares (for example the amazon plugin for amazon-ebs builds). You run it once per template (or whenever the plugin requirements change) before packer validate or packer build, typically as the first step of a CI pipeline.

Q: Why run packer validate as a separate CI step before packer build?

A: validate checks the template's syntax and configuration without launching anything, so it fails fast - in seconds - on a typo or misconfiguration. build is far more expensive: it launches a real temporary instance in the target cloud. Catching mistakes at validate time avoids paying for and waiting on a build that was always going to fail.

Q: How would you build a CI/CD pipeline that produces a new AMI automatically?

A: Store the HCL template and provisioning scripts in version control alongside a workflow (GitHub Actions, Jenkins, etc.) triggered on commit. Store cloud credentials as pipeline secrets, never in the template. The pipeline installs Packer, runs packer init, packer validate, then packer build; the build step launches a temporary instance, runs the provisioner, registers the AMI, and tears the instance down automatically. The new AMI ID is then available to Terraform, a launch template, or an Auto Scaling group as the next version to deploy.

Q: Where do secrets like AWS access keys belong in a Packer + GitHub Actions pipeline, and why?

A: In the repository's encrypted Actions secrets (Settings -> Secrets and variables -> Actions), never hardcoded in the HCL template or committed to the repo. The workflow injects them as environment variables at build time, so they never appear in source control or in build logs, and they can be rotated or scoped to least-privilege independently of the template itself.

Q: Can one Packer configuration produce images for multiple cloud platforms at once?

A: Yes - a single Packer configuration can define multiple source/builder blocks (amazon-ebs, azure-arm, googlecompute, vmware-iso, etc.) inside one build block, so one packer build run can fan out and produce an AWS AMI, an Azure Managed Image, a Google Compute Image, and a VMware Template from the same provisioning scripts in a single pass.

Q: How does a Packer-built AMI fit into a larger multi-account AWS setup, like a Landing Zone?

A: A dedicated shared-services/tooling account is a natural place to run the Packer pipeline and own the golden AMIs; the resulting AMI can then be shared or copied to workload accounts (Dev/Stage/Prod) for consumption in launch templates or Terraform. This keeps image-building permissions and the CI credentials isolated from the workload accounts that actually run production traffic.